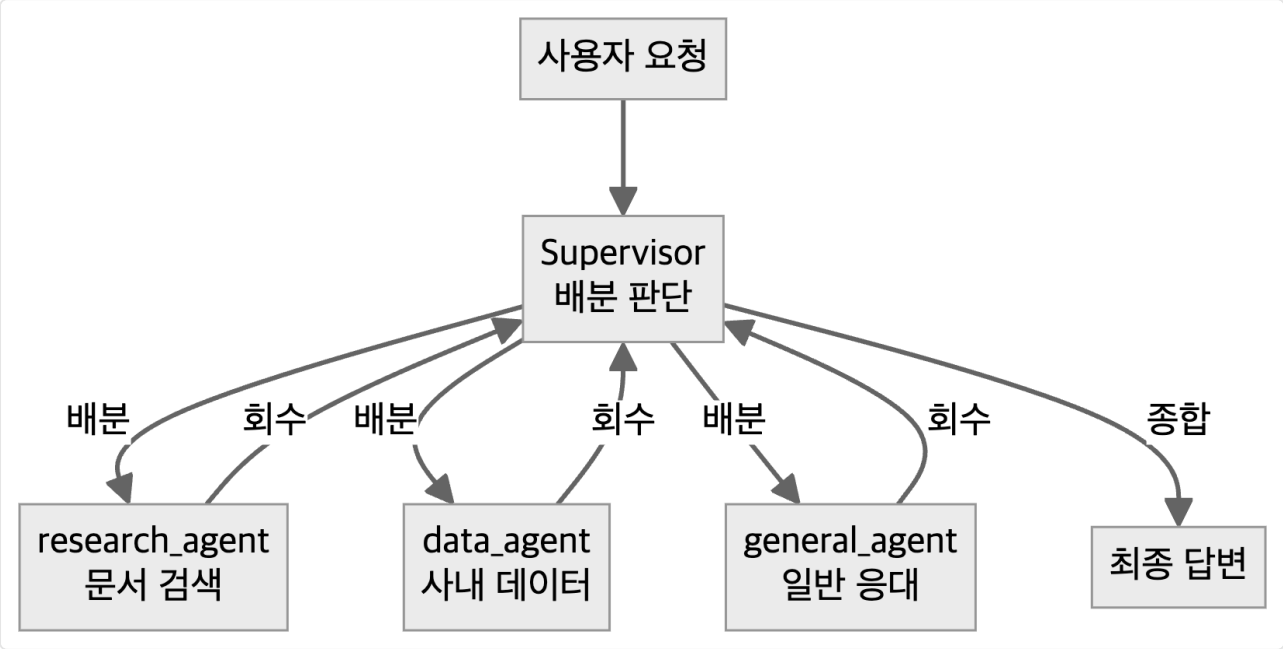
Supervisor 패턴의 구조 (실습)

2-1. Supervisor의 역할
2-2. 전문 Agent 설계
2-3. 라우팅 동작 관찰 (실습)
2-4. 다른 방식: Agent를 도구로 노출하기 (agents-as-tools)

2-1. Supervisor의 역할

> Supervisor의 라우팅 결정이 결국 분류 문제라는 것을 코드로 확인해봅니다.

Supervisor 패턴의 실행 흐름은 루프입니다. 모든 흐름이 Supervisor를 거칩니다.



핵심은 세 가지입니다.

- **루프의 중심**: Supervisor가 흐름의 중심입니다.
- **하위 Agent의 복귀**: 각 하위 Agent는 작업을 끝내면 결과를 messages에 남기고 Supervisor로 돌아갑니다.
- **다음 행동 결정**: Supervisor는 대화 기록을 보고 다음에 누구를 부를지 아니면 끝낼지 결정합니다.

Supervisor의 결정은 결국 모델 호출 한 번입니다. 지금까지의 대화를 보고 다음에 호출할 Agent 이름을 출력하라는 분류 문제를 푸는 것입니다.

그래서 Supervisor 프롬프트는 라우팅 기준만 짧고 명확하게 쓰는 것이 중요합니다. Supervisor에게 도메인 지식까지 가르치려 하면 안 됩니다.

따라하기

1. Agent 없이 라우터만 만들어 결정 과정을 관찰합니다. 1일차 구조화 출력을 그대로 씁니다. 아래 코드를 `router_only.py` 로 저장해 실행합니다.

```
# router_only.py - Supervisor의 핵심인 라우팅 결정만 분리해 관찰
from typing import Literal
from dotenv import load_dotenv
from pydantic import BaseModel, Field
from langchain_aws import ChatBedrockConverse

load_dotenv()

class Route(BaseModel):
    next_agent: Literal["data_agent", "research_agent", "general_agent", "FI
NISH"] = Field(
        description="다음에 호출할 Agent, 또는 작업 완료 시 FINISH"
    )
    reason: str = Field(description="선택 이유 한 줄")

llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
    temperature=0,
)
router_llm = llm.with_structured_output(Route)

ROUTING_GUIDE = (
    "다음 질문을 처리할 Agent를 골라라.\n"
    "- data_agent: 임직원, 자산, 프로젝트 등 사내 시스템 데이터 조회\n"
    "- research_agent: 연차, 재택근무, 출장 등 사내 규정 문서 검색\n"
    "- general_agent: 인사말, 잡담, 범위 밖 질문 응대\n"
    "- FINISH: 이미 답변 가능\n\n질문: "
)

questions = [
    "클라우드운영팀 임직원 명단 알려줘",
    "연차는 며칠까지 쓸 수 있어?",
    "안녕하세요, 오늘 기분 어때요?",
]
for q in questions:
    decision = router_llm.invoke(ROUTING_GUIDE + q)
    print(f"{q}\n  -> {decision.next_agent} ({decision.reason})\n")
```

2. 라우팅 안내문에서 담당 설명을 "문서 관련"처럼 모호하게 바꿔 다시 실행하고 정확도가 떨어지는 것을 확인합니다.

막힐 때

- `with_structured_output` 오류: 모델 ID가 도구 호출을 지원하는 모델인지 확인합니다. Bedrock의 Claude 계열은 지원합니다.
- 라우팅이 계속 틀립니다: 각 Agent의 담당을 명사 나열로 구체화합니다. "문서 관련"보다 "연차, 재택근무, 출장 등 사내 규정 문서"가 훨씬 정확합니다. 라우팅 품질은 대부분 이 안내문 품질에서 결정됩니다.

확인: 세 질문이 각각 data_agent, research_agent, general_agent로 배분되고 선택 이유가 함께 출력되면 성공입니다.

> 라우팅 품질이 배분 기준 문구의 구체성에서 나온다는 걸 확인했습니다.

2-2. 전문 Agent 설계

> 담당 범위, 도구, 프롬프트를 좁게 유지하는 설계 원칙을 이해해봅니다.

전문 Agent 설계의 제1원칙은 좁게 유지하기입니다. 세 가지를 좁게 잡습니다.

1. 담당 범위를 좁게 잡습니다. 사내 데이터 조회 Agent는 문서 질문을 받아도 문서를 검색하지 않습니다. 자기 범위 밖이면 담당이 아니라고 답하고 돌아가면 됩니다. 범위 밖 처리는 Supervisor의 몫입니다.
2. 도구를 좁게 줍니다. 각 Agent에는 그 도메인의 도구만 줍니다. 데이터 Agent에 문서 검색 도구를 혹시 몰라서 얹는 순간 1번째 세션 1-1의 혼동 문제가 Agent 안에서 재발합니다.
3. 프롬프트를 좁게 씁니다. 무엇 전문가이고 무엇만 하며 무엇은 하지 않는지 정도로 짧게 씁니다. 5줄을 넘기면 설계를 의심합니다.

오늘 만들 전문 Agent 3종의 설계표입니다.

Agent	담당 범위	도구	모델 선택
research_agent	연차, 재택근무, 출장 등 사내 규정 문서	retrieve_docs (2일차 RAG)	가벼운 모델로 충분합니다
data_agent	임직원, 자산, 프로젝트 등 사내 시스템 데이터	search_employee, list_team_members, get_asset, list_projects (4일차 MCP)	가벼운 모델로 충분합니다
general_agent	인사말, 잡담, 범위 밖 안내	없음	가벼운 모델로 충분합니다
supervisor	배분과 종합	하위 Agent 자체	결정 정확도가 중요하므로 상위 모델을 씁니다

모델을 Agent별로 다르게 쓸 수 있다는 것이 Multi-Agent의 실질적인 장점입니다. 조회와 검색은 가벼운 모델로 충분하고 배분 결정과 최종 종합은 상위 모델에 맡깁니다.

일반 응대 Agent를 따로 두는 이유도 짚고 갑니다. 인사말이나 범위 밖 질문까지 데이터 Agent로 보내면 그 Agent가 불필요한 도구 호출을 하거나 어색한 답을 만듭니다. 그 외 전부를 받아주는 Agent가 있으면 배분 실패의 안전망이 됩니다.

따라하기

1. 아래 체크리스트를 실습 폴더의 메모에 옮겨 적고 3번째 세션 3-1에서 Agent를 만들 때마다 항목을 확인합니다.

```
[ ] 이 Agent의 담당을 한 문장으로 말할 수 있는가
[ ] 도구 목록에 다른 도메인의 도구가 섞여 있지 않은가
[ ] 시스템 프롬프트가 5줄 이내인가
[ ] Supervisor의 배분 기준에 이 Agent의 담당이 명사로 구체적으로 적혀 있는가
[ ] 범위 밖 질문이 왔을 때의 행동(거절하고 복귀)이 프롬프트에 있는가
```

막힐 때

- Agent가 범위 밖 질문에도 열심히 답합니다: 시스템 프롬프트에 담당 범위 밖 질문에는 답하지 말고 범위 밖이라고만 답하라를 명시합니다. 하위 Agent가 아는 척을 하면 Supervisor가 다시 배분할 기회를 잃습니다.
- 도구 없는 Agent도 만들 수 있습니까: 됩니다. general_agent처럼 도구 목록을 빈 리스트로 두면 순수 대화 Agent가 됩니다. Supervisor에 넣을 때 name만 잊지 않습니다.

확인: 다섯 항목 체크리스트를 손에 들고 3번째 세션 3-1로 넘어갈 준비가 되면 성공입니다.

> 전문 Agent는 담당, 도구, 프롬프트를 모두 좁게 유지한다는 원칙을 익혔습니다.

2-3. 라우팅 동작 관찰 (실습)

> 가장 작은 2-Agent Supervisor를 만들어 배분과 회수 과정을 눈으로 확인해봅니다.

본격 구현 전에 하위 Agent 2개짜리 Supervisor를 만들어 배분 과정을 확인합니다. 여기서 `langgraph-supervisor` 라이브러리를 처음 사용합니다.

`create_supervisor` 는 두 가지를 자동으로 만들어 줍니다. 하나는 하위 Agent들을 노드로 배치하고 각 Agent가 끝나면 Supervisor로 돌아오는 연결입니다.
다른 하나는 Supervisor가 하위 Agent를 호출하는 `transfer_to_이름` 형태의 handoff(작업을 다른 Agent에게 넘기는 것) 도구입니다.
그래서 우리는 하위 Agent 목록, Supervisor 모델, 배분 기준 프롬프트만 넘기면 됩니다.

따라하기

1. 더미 도구를 가진 Agent 2개를 만들어 Supervisor로 묶고 실행 과정을 스트리밍으로 관찰합니다. 아래 코드를 `two_agent_supervisor.py`로 저장해 실행합니다.
   코드 앞부분의 `get_text`는 응답에서 텍스트만 뽑아내는 헬퍼입니다. Bedrock 응답의 `content`가 문자열이 아니라 블록 리스트로 올 때가 있기 때문입니다.
   도구 호출이 오가는 Multi-Agent에서는 이 형태가 실제로 나오므로 메시지 내용을 출력할 때마다 이 헬퍼를 거칩니다.

```
# two_agent_supervisor.py — 최소 2-Agent Supervisor로 배분 관찰
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage
from langgraph.prebuilt import create_react_agent
from langgraph_supervisor import create_supervisor

load_dotenv()
MODEL = "us.anthropic.claude-sonnet-4-5-20250929-v1:0"

# 더미 도구지만 이름과 인자는 4일차 MCP 서버와 똑같이 맞춰 둡니다
@tool
def list_team_members(team: str) -> str:
    """팀 이름으로 소속 임직원 목록을 조회한다."""
    data = {
        "클라우드운영팀": "클라우드운영팀 3명: 김하늘(팀장), 이서준(인프라 엔지니어), 정하은(데이터 분석가)",
        "물류플랫폼팀": "물류플랫폼팀 2명: 박도윤(백엔드), 최민준(프론트엔드)",
    }
    return data.get(team, f"'{team}' 팀을 찾을 수 없습니다.")

@tool
def retrieve_docs(query: str) -> str:
    """사내 규정 문서를 검색한다."""
    return f"검색 결과 [{query}]: 연차는 연 15일 부여, 입사 1년 미만은 매월 1일씩 최대 11일"

worker_llm = ChatBedrockConverse(model=MODEL, region_name="us-east-1", temperature=0)

def get_text(message):
    """ChatBedrockConverse는 content를 블록 리스트로 주기도 하므로 텍스트만 모아 반환합니다."""
    content = message.content
    if isinstance(content, list):
        return "".join(block.get("text", "") for block in content if isinstance(block, dict))
    return content

# 하위 Agent에는 name이 필수입니다. 이 이름으로 handoff 도구가 만들어집니다
data_agent = create_react_agent(
    worker_llm,
    [list_team_members],
    prompt="너는 사내 데이터 조회 전문가다. 임직원 데이터 조회만 담당한다.",
    name="data_agent",
)
```

```
research_agent = create_react_agent(
    worker_llm,
    [retrieve_docs],
    prompt="너는 사내 규정 문서 검색 전문가다. 문서 검색만 담당한다.",
    name="research_agent",
)

supervisor_llm = ChatBedrockConverse(model=MODEL, region_name="us-east-1", t
emperature=0)
supervisor = create_supervisor(
    [data_agent, research_agent],
    model=supervisor_llm,
    prompt=(
        "너는 삼성SDS 사내 어시스턴트의 작업 분배자다.\n"
        "- 임직원 등 사내 데이터 조회는 data_agent\n"
        "- 연차 등 규정 문서 검색은 research_agent\n"
        "필요한 Agent를 순서대로 호출하고 모두 끝나면 사용자에게 답하라."
    ),
)
app = supervisor.compile()

def run(question: str):
    print(f"질문: {question}")
    for event in app.stream(
        {"messages": [HumanMessage(content=question)]},
        stream_mode="updates",
        config={"recursion_limit": 25},  # 무한 루프 안전장치
    ):
        for node, update in event.items():
            for m in (update or {}).get("messages", []):
                if getattr(m, "tool_calls", None):
                    for tc in m.tool_calls:
                        print(f"  [{node}] 도구 호출: {tc['name']}({tc.get('ar
gs', {})})")
                elif getattr(m, "content", None):
                    label = getattr(m, "name", None) or node
                    print(f"  [{label}] {get_text(m)[:120]}")
    print()

run("클라우드운영팀 임직원 명단 알려줘")
run("연차는 며칠이야?")
```

2. 출력에서 `transfer_to_data_agent` 같은 도구 호출을 찾아 표시해 둡니다. Supervisor 입장에서 하위 Agent 호출이 handoff 도구를 고르는 일이라는 것을 확인합니다.

막힐 때

- `agent must have a name` 오류: `create_react_agent`에 `name=`을 빠뜨린 경우입니다. Supervisor는 Agent 이름으로 handoff 도구를 만들기 때문에 name이 필수입니다. 구버전이라 name 인자를 받지 않으면 만든 뒤에 `data_agent.name = "data_agent"`로 지정합니다.
- 스트리밍 출력에서 update가 None입니다: 노드에 따라 비어 있을 수 있습니다. 위 코드처럼 `(update or {})`로 방어합니다.
- Supervisor가 하위 Agent를 안 부르고 직접 답합니다: 프롬프트에 직접 답하지 말고 반드시 담당 Agent를 통해 확인하라를 추가합니다.
- 같은 Agent를 계속 반복 호출합니다: 하위 Agent의 답이 모호하면 Supervisor가 아직 안 끝났다고 판단합니다. 하위 Agent 프롬프트에 조회 결과를 명확한 한 문단으로 정리해 보고하라를 추가하면 대부분 해결됩니다. `recursion_limit`을 걸어 두는 습관도 함께 들입니다.

확인: 첫 질문에서는 data_agent만 호출되고 두 번째 질문에서는 research_agent만 호출되며 각각 Supervisor가 최종 답을 다시 정리하면 성공입니다.

> Supervisor의 배분이 handoff 도구 호출로 나타나고 결과가 다시 Supervisor로 회수된다는 걸 확인했습니다.

2-4. 다른 방식: Agent를 도구로 노출하기 (agents-as-tools)

> Multi-Agent를 구성하는 또 하나의 방식을 알고 언제 어느 쪽을 고를지 판단해봅니다.

지금까지 본 Supervisor 방식은 **제어권을 넘깁니다.** handoff 도구가 불리면 하위 Agent가 대화(messages) 자체를 이어받아 일하고 끝나면 Supervisor로 돌아옵니다.

다른 방식도 있습니다. 하위 Agent를 그냥 **도구로 감싸서** 상위 Agent가 함수처럼 부르는 것입니다. 제어권은 상위가 계속 쥐고 있고 하위는 질문 문자열을 받아 답 문자열만 돌려줍니다.

```
# research_agent를 도구로 감싸면 끝입니다
@tool
def ask_research(question: str) -> str:
    """연차, 재택근무, 출장 등 사내 규정 문서를 찾아 답한다."""
    r = research_agent.invoke({"messages": [HumanMessage(question)]})
    return get_text(r["messages"][-1])
```

둘의 차이는 이렇습니다.

비교	Supervisor (handoff)	agents-as-tools
제어권	하위 Agent로 넘어감	상위가 계속 쥐고 있음
대화 맥락	전체 messages를 공유	넘긴 질문 문자열만 전달
여러 Agent 동시 호출	순차적이 기본	도구 병렬 호출로 쉬움
구조 복잡도	높음 (라우팅 루프)	낮음 (도구 목록에 하나 추가)
잘 맞는 상황	여러 턴 협업, 중간 결과를 보고 다음을 정해야 할 때	하위가 단발성 조회·요약으로 끝날 때

오늘은 Supervisor 방식으로 진행합니다. 최종 시나리오가 "규정을 찾고 그 값으로 데이터를 조회해 종합"처럼 중간 결과를 보고 다음을 정해야 하는 구조이기 때문입니다. 다만 사내에서 간단한 조합만 필요하다면 Supervisor까지 갈 것 없이 도구로 감싸는 쪽이 더 쉬운 선택일 수 있습니다.

다른 프레임워크도 같은 분류를 씁니다. Microsoft Agent Framework 문서의 agents-as-tools 설명을 함께 보면 프레임워크와 무관한 공통 패턴이라는 것을 확인할 수 있습니다.

https://learn.microsoft.com/en-us/agent-framework/journey/agents-as-tools

> 제어권을 넘길지 쥐고 있을지가 두 방식의 갈림길이라는 걸 확인했습니다.